GeeksFinder — WhatsApp alerts & admin panel: PRD, TRD & database spec

Oct 19, 2026 · @digidiploma

1. Scope and how to use this doc

This spec adds three things to GeeksFinder: WhatsApp morning alerts as a paid "Pro + WhatsApp" plan at ₹29 per 15 days, a full admin panel that covers operations, money, marketing and the visit-to-paid funnel, and a content quality bot that auto-publishes only low-risk blog articles that pass every check (section 10). It builds on the business plan, DESIGN.md and the LinkedIn spec, and changes nothing in them except where it says so.

How to hand this to Claude Code:

1. Export this doc as Markdown and save it as `docs/whatsapp-admin/SPEC.md` in the repo.
2. Add two lines to `CLAUDE.md`: "WhatsApp and admin work follows docs/whatsapp-admin/SPEC.md. The hard rules in its section 1 and the security rules in its section 7 override anything else."
3. Approve the DESIGN.md additions in section 8 first, because CLAUDE.md says UI not covered by DESIGN.md must be asked about.
4. Do the manual Meta and Razorpay setup in section 3 before PR 3.
5. Run the prompts in section 9 one at a time, in order. Review and merge each before the next.

#	Feature	What it gives	Priority
W1	WhatsApp opt-in and verification	Student adds a WhatsApp number, gives consent, verifies with a code	P0
W2	Daily WhatsApp digest	One message each morning with top matches, only on days with new matches	P0
W3	Pro + WhatsApp plan	₹29 per 15 days by UPI Autopay; upgrade and downgrade	P0
W4	Inbound and status handling	STOP opt-out, one auto-reply, delivery and cost tracking	P0

#	Feature	What it gives	Priority
A1	Admin foundation	Roles, audit log, kill switches, spend caps, admin alerts	P0
A2	Overview, users, payments, system	Daily numbers vs targets, user lookup and actions, billing health	P0
A3	Operations	Jobs, matching, alerts, crons, AI cost, WhatsApp health	P0
A4	Product modules	Tailoring, HR outreach, do-not-contact, outcomes, LinkedIn	P1
A5	Marketing and funnel	Event tracking, UTM attribution, funnel, ambassadors, referrals, announcements, Telegram channel, cohorts, feedback	P1
A6	Content and SEO	Article queue (existing), job hubs, Search Console numbers	P2

Hard rules for WhatsApp (copy these into code review):

- At most one business-initiated template message per student per IST day, and only when there is at least one new match. No free-form chat, no marketing broadcasts, ever.
- Send only when all of these hold: plan entitles WhatsApp, number verified, consent recorded, not opted out, not paused, the `whatsapp_sends` switch is on, and the month's WhatsApp spend is under its cap.
- Use Meta's WhatsApp Cloud API directly. Only `web/src/lib/whatsapp/client.ts` may call `graph.facebook.com`. The token lives only in server env vars.
- Message buttons are URL buttons into GeeksFinder. No quick-reply buttons.
- Every webhook request is verified with `X-Hub-Signature-256` before anything is read.
- Phone numbers are masked (98xxxxxx21) in the admin panel and logs, and never exported in bulk.

Hard rules for the admin panel:

- Every admin page and `/api/admin/*` route checks the role on the server from the database and returns 404 to anyone else.
- Every admin action that changes data writes one `audit_log` row with before and after values.

- Kill switches and spend caps are enforced inside the code that sends or spends, not only shown in the UI.
- No admin screen shows resume text, raw phone numbers, Gmail tokens or payment card data.
- Real numbers only. A metric with no data shows "No data yet", never a placeholder number.

2. WhatsApp PRD

Students who live on WhatsApp get the same morning digest there for ₹29 per 15 days, a price that stays profitable even if Meta bills every alert at its marketing rate.

Pricing

Plan	Price	Alerts
Free	₹0	Weekly, Telegram and email
Pro	₹10 per 15 days	Daily, Telegram and email
Pro + WhatsApp	₹29 per 15 days	Everything in Pro, plus the daily WhatsApp digest

Cost basis (Meta India rate card, as of Oct 2026): marketing ₹0.8631 and utility ₹0.115 per delivered message, plus 18% GST (source, source). One daily digest for 15 days costs ₹15.28 at the marketing rate or ₹2.04 at the utility rate. After about 2.4% Razorpay fees (₹0.70), the margin per cycle is about ₹13 in the worst case and ₹26 in the best. Rates live in the `wa_rate_card` table, never in code.

Goals

- Give WhatsApp-first students the daily alert habit on the app they open most.
- Keep a positive margin on every WhatsApp user, every cycle, at the marketing rate.
- Keep the number's quality rating green and the monthly opt-out rate under 2% (proposed).
- Targets by day 30 after release (proposed): 10% of Pro users upgrade, 70% of WhatsApp digests are read.

Non-goals

- Chat or support over WhatsApp.
- Marketing broadcasts, offers or announcements on WhatsApp.
- WhatsApp for Free users, in any form.
- WhatsApp login or OTP sign-in to GeeksFinder.
- Sending HR emails, LinkedIn notes or follow-up reminders on WhatsApp. Those stay on Telegram and email.

User stories

1. As a Pro student, I tap "Add WhatsApp alerts", see "₹29 every 15 days", and authorise a new UPI mandate.
2. As a new student, I can pick Pro + WhatsApp directly at checkout, with the same 7-day trial.
3. As a student, I enter my WhatsApp number, tick the consent box, get a 6-digit code on WhatsApp, enter it, and see "WhatsApp connected".
4. As a student, I get one WhatsApp message at my alert time with my top new matches and two buttons: "See matches" and "Tailor top match".
5. As a student, I get no message on days without new matches.
6. As a student, I reply STOP and alerts stop at once, with one confirmation message.
7. As a student, I turn WhatsApp off in Alerts, or switch back to Pro from the next cycle.
8. As a student who replies with a question, I get one fixed reply pointing me to Help in the app, at most once a day.

Functional requirements

ID	Requirement	Priority
W1.1	Phone field (India +91 default, stored as E.164) and consent checkbox; consent time and copy version stored	P0
W1.2	Verification with a WhatsApp authentication template: 6 digits, 10-minute expiry, 3 wrong tries, 3 sends per hour	P0
W1.3	A number can be verified on only one account	P0

ID	Requirement	Priority
W2.1	Daily digest template with up to 3 job lines, match count and 2 URL buttons	P0
W2.2	Sent at the student's alert time in IST, outside quiet hours; skipped when no new matches; one per IST day even if the cron runs twice	P0
W2.3	Channels list gains `whatsapp`; a student can pick WhatsApp only, or WhatsApp plus Telegram and email	P0
W2.4	After 3 failed deliveries in a row, pause WhatsApp for that student and show a banner and email asking them to check the number	P0
W3.1	Razorpay plan `pro_wa`: 2900 paise, every 15 days, UPI Autopay, 7-day trial for new subscribers	P0
W3.2	Upgrade from Pro: new mandate; WhatsApp starts once the mandate is authorised; ₹29 debits start when the current Pro cycle ends	P0
W3.3	Downgrade: Pro + WhatsApp runs to the end of the paid cycle, then Pro starts on a new ₹10 mandate	P1
W4.1	Status webhook stores sent, delivered, read and failed, with Meta's billed category and the cost	P0
W4.2	STOP, UNSUBSCRIBE or CANCEL opts out at once; START opts back in if the plan allows	P0
W4.3	Any other inbound text is stored and gets one fixed auto-reply at most once per 24 hours	P0
W4.4	Plan limits (tailoring, LinkedIn notes, WhatsApp) are read from the `plan_limits` table	P0

Acceptance criteria

- A Pro student upgrades, verifies a number and gets the next morning's digest on WhatsApp; their next debit shows ₹29.

- Running the notify cron twice in one morning sends exactly one WhatsApp digest per student.
- A student with zero new matches gets no WhatsApp message that day.
- Replying STOP stops the next day's digest; the Alerts page shows WhatsApp off.
- With the `whatsapp_sends` switch off, or the monthly cap reached, no WhatsApp message is sent and the admin gets one alert.
- A forged webhook without a valid signature changes nothing and returns 401.
- A free user cannot verify a number or receive any WhatsApp message, even by calling the API directly.

Open questions

- Confirm ₹29 per 15 days before Razorpay plan creation. A UPI mandate amount cannot be raised later without every user authorising a new mandate.
- WhatsApp during the 7-day trial costs up to ₹7.13 per trial user at the marketing rate. Proposed: include it, since the trial is where students form the habit.
- On upgrade, WhatsApp runs free until the current Pro cycle ends, up to about ₹14 per upgrade. Proposed: accept it as an acquisition cost.
- Meta decides the template category. We submit the digest as utility; if Meta sets it to marketing, the price above still holds.

3. WhatsApp TRD

WhatsApp runs inside the existing Next.js app, Supabase and the GitHub Actions notify cron, talking to Meta's Cloud API directly with no reseller in between.

Setup (one time, manual)

1. Verify the business in Meta Business Manager.
2. Create a Meta app, add the WhatsApp product, and register a dedicated phone number that is not used in the WhatsApp app. Request the display name "GeeksFinder".
3. Create a system user with a permanent token and the `whatsapp_business_messaging` and `whatsapp_business_management` permissions.
4. Set server env vars: `WHATSAPP_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_WABA_ID`, `WHATSAPP_APP_SECRET`, `WHATSAPP_VERIFY_TOKEN`. Never `NEXT_PUBLIC_*`.

5. Set the webhook URL to `https://<domain>/api/whatsapp/webhook` and subscribe to the `messages` field.
6. Submit the templates below and wait for approval.
7. In Razorpay, create plan `pro_wa`: period `daily`, interval 15, amount 2900 paise (fallback: weekly × 2, as for Pro).

Templates

Template variables cannot contain newlines, so each job is its own variable and each line is cut to 60 characters with newlines and tabs removed.

Name	Category	Body	Buttons
`gf_verify`	Authentication	Standard 6-digit code	Copy code
`gf_daily_3`	Utility (submitted)	"Good morning {{1}}, you have {{2}} new matches today." then 3 lines "{{3}}", "{{4}}", "{{5}}", then "Open GeeksFinder to tailor your resume for any of them." Footer: "Reply STOP to turn off"	URL "See matches" → `/m/{{1}}`; URL "Tailor top match" → `/j/{{1}}`
`gf_daily_1`	Utility (submitted)	Same, with 1 job line; used when there are 1 or 2 new matches	Same

A job line looks like "Frontend Developer Intern · Zepto · Bengaluru · 87%". Button suffixes are the digest id and the top job id; the app pages behind them require login.

New and changed files

Path	Purpose
`web/src/lib/whatsapp/client.ts`	`sendTemplate()`; the only code allowed to call `graph.facebook.com`; checks the switch and spend cap before every send

Path	Purpose
`web/src/lib/whatsapp/phone.ts`	zod E.164 validator (India default) and `maskPhone()`
`web/src/lib/whatsapp/templates.ts`	Template names and parameter builders with the 60-character, no-newline sanitiser
`web/src/lib/whatsapp/cost.ts`	Rate lookup from `wa_rate_card`; cost per message with GST
`web/src/lib/whatsapp/eligibility.ts`	`canSendWhatsApp(user)`: plan, verified, consent, not opted out, not paused, switch, cap
`web/src/app/api/whatsapp/verify/start/route.ts`	Save number and consent, send code
`web/src/app/api/whatsapp/verify/confirm/route.ts`	Check code, mark verified
`web/src/app/api/whatsapp/disconnect/route.ts`	Turn WhatsApp off for the student
`web/src/app/api/whatsapp/webhook/route.ts`	GET verify challenge; POST statuses and inbound messages
`web/src/app/api/billing/whatsapp/upgrade/route.ts`	Create the `pro_wa` subscription
`web/src/app/api/billing/whatsapp/downgrade/route.ts`	Schedule the switch back to Pro
`web/src/app/api/webhooks/razorpay/route.ts`	Existing; handle `pro_wa` events
`web/scripts/notify/whatsapp.ts`	`sendWhatsAppDigest(user, matches)` called by the existing notify cron
`web/scripts/admin/whatsapp-health.ts`	Daily snapshot of quality rating, limit tier and template status
`web/supabase/migrations/<ts>_whatsapp_admin.sql`	Section 6

Verification

`POST /api/whatsapp/verify/start` body `{ phone, consent: true }`:

1. `requireUser`, zod, rate limit (3 per hour per user and per number).
2. Refuse with 403 unless `plan_limits.whatsapp` is true for the student's effective plan.
3. Refuse with 409 if another account has this number verified.
4. Upsert `whatsapp_contacts` with `status = 'pending'`, `consent_at`, `consent_version`.
5. Create a 6-digit code, store only its SHA-256 hash in `whatsapp_otps` with a 10-minute expiry, send `gf_verify`.

`POST /api/whatsapp/verify/confirm` body `{ code }`: compare hashes in constant time, allow 3 tries, then set `status = 'verified'`, `verified_at`, clear `opted_out_at` and `paused`.

Consent copy (versioned as `wa-consent-v1`): "Send my daily job matches to this WhatsApp number. One message each morning, only when there are new matches. Reply STOP anytime."

Daily digest

The existing notify cron already builds each student's digest. After it does, for students whose channels include `whatsapp`:

1. `canSendWhatsApp(user)`; skip silently if false, except the switch and cap, which log one `cron_runs` warning per run.
2. Skip if the digest has no new matches.
3. Insert a `wa_messages` row with `kind = 'digest'` and `send_date` = today in IST. The unique index makes a second run skip.
4. Pick `gf_daily_3` for 3 or more new matches, else `gf_daily_1`. Fill parameters from the top matches by score.
5. Call `sendTemplate()` at no more than 20 messages a second. Store the Meta message id and the estimated cost at the template's submitted category.
6. On an API error, mark the row failed with the error code and increment `fail_streak`.

Webhook

- GET: return `hub.challenge` only if `hub.verify_token` equals `WHATSAPP_VERIFY_TOKEN`.

- POST: read the raw body, verify `X-Hub-Signature-256` (HMAC-SHA256 with `WHATSAPP_APP_SECRET`, constant-time compare), else 401. Return 200 quickly and process in the same request with a 5-second budget.
- Statuses: update `wa_messages` by Meta message id; store status time, error code, and the billed `category` and `billable` flag from the status's pricing data; recompute `cost_inr` from the rate card. Delivered resets `fail_streak`; 3 failures in a row set `paused = true`.
- Inbound text: store in `wa_inbound` (trimmed to 1,000 characters). STOP, UNSUBSCRIBE or CANCEL (any case) sets `opted_out_at` and sends one confirmation. START clears it if the plan allows. Anything else gets the fixed auto-reply if `last_auto_reply_at` is over 24 hours ago: "This number only sends GeeksFinder job alerts. For help, open GeeksFinder and tap Help."
- Every event is idempotent by Meta message id.

Billing

- `subscriptions` gains `plan_code` (`pro` or `pro_wa`). The student's effective plan is the highest of an active subscription and an active `pro_grants` row.
- Upgrade: create a `pro_wa` subscription with `start_at` = the current Pro `current_period_end` (or now plus 7 days for a new trial), notes `{ user_id, from_plan }`. On `subscription.authenticated`, allow WhatsApp and set the old Pro subscription to cancel at cycle end.
- `subscription.activated` and `subscription.charged` on `pro_wa` set `plan = 'pro_wa'`. `halted` sets `free` and stops WhatsApp the same day. `cancelled` keeps WhatsApp until `current_period_end`.
- Downgrade: cancel `pro_wa` at cycle end and create a `pro` subscription starting then; the student authorises the ₹10 mandate.

API summary

All errors return `{ error: { code, message } }` with a plain message that says what happened and how to fix it.

Method + path	Auth	Rate limit	Returns
POST `/api/whatsapp/verify/start`	requireUser	3/h	`{ status: 'pending' }`
POST `/api/whatsapp/verify/confirm`	requireUser	10/h	`{ status: 'verified' }`
POST `/api/whatsapp/disconnect`	requireUser	general	`{ ok }`
GET, POST `/api/whatsapp/webhook`	Meta signature	none	200
POST `/api/billing/whatsapp/upgrade`	requireUser	general	`{ subscription_id, short_url }`
POST `/api/billing/whatsapp/downgrade`	requireUser	general	`{ ends_at }`

4. Admin panel PRD

The admin panel lets one founder run GeeksFinder in 10 minutes a day: see the funnel and the money, catch failures before students do, and fix things in one click with every action logged.

Goals

- One page that answers "is everything working, and are we growing?" every morning.
- Stop any sending or spending in one click when something breaks.
- Know which marketing channel, college and ambassador brings students who pay and stay.
- Track the plan's success metrics against their targets without spreadsheets.

Non-goals

- Logging in as a student.
- Editing a student's resume, matches or tracker.
- A general BI tool; the panel shows fixed metrics, not ad-hoc queries.
- Third-party analytics scripts inside the app pages.

Roles

Role	Can see	Can do
Owner	Everything	Everything, including roles, flags, spend caps and plan limits
Ops	Overview, users, jobs, alerts, AI, WhatsApp, outreach, payments (read)	User actions, hide jobs, do-not-contact, rerun crons, pause a channel
Marketing	Overview, funnel, acquisition, ambassadors, referrals, campaigns, Telegram channel, cohorts, feedback, SEO	Ambassador codes, rewards up to 30 Pro days, campaigns, channel posts
Reviewer	Content queue only	Approve, edit, reject articles

Modules

ID	Module	Shows	Actions	Priority
A1	Overview	Today vs targets: signups, activation, weekly active, Free → Pro, autopay success, tailored per active user; open problems list	Jump to the problem	P0
A2	Users	Search by email or name; plan, usage this cycle, channels, tracker counts, attribution, masked phone	Grant Pro days, reset quota, turn off a channel, disable account, run data deletion	P0
A3	Jobs and ingestion	Per source: fetched today, errors, API quota left, dedupe rate, fresher share, via LinkedIn share; enrich failures; student reports	Add or remove company boards, hide a job, mark expired	P0

ID	Module	Shows	Actions	Priority
A4	Matching	Score distribution, matches per user, users with zero matches and the likely reason	None	P1
A5	Alerts and crons	Per channel sent, delivered, failed; Telegram blocks; email bounces and unsubscribes; every cron run with status, duration, counts	Rerun a failed cron, pause a channel	P0
A6	WhatsApp	Messages and cost today and this month, cost per WhatsApp user vs ₹29, spend vs cap, template status and category, quality rating, limit tier, delivery funnel, opt-outs, inbound replies with free-service counter	Pause all sends, turn off a student's WhatsApp	P0
A7	AI usage and cost	Calls per task, Gemini → Groq fallback rate, invalid JSON retries, failures, Claude article spend vs cap, 429 counts	Pause AI generation	P0
A8	Resume tailoring	Tailored count, share of changes accepted vs undone, PDF download rate	None	P1
A9	HR outreach	Finder hit rate per chain step, Hunter and Apollo credits left, drafted vs sent, Gmail cap hits, reminders sent	Add, remove, search do-not-contact; log HR removal requests	P0
A10	Outcomes	Applications by stage; interviews and offers reported	None	P1

ID	Module	Shows	Actions	Priority
A11	Payments	Plan mix, MRR, new and cancelled, failed debits, halted, upgrade mandates started but not authorised, trials ending this week, Razorpay webhook log	Resend a payment reminder	P0
A12	Funnel and acquisition	The full funnel below, by source, campaign, ambassador, college, city and signup week	Export aggregate CSV (no personal data)	P1
A13	Ambassadors and referrals	Codes, signups, paid users and rewards per ambassador; referral invites, qualified, rewarded	Create codes, grant rewards, block a fraudulent referral	P1
A14	Campaigns and Telegram channel	Email and Telegram announcements by segment with sent counts; daily channel post queue; channel clicks to the bot	Draft, schedule, send; edit or skip a channel post	P1
A15	Retention and feedback	Weekly retention by signup week; feedback inbox by tag; top 3 complaints this week	Tag, mark fixed	P1
A16	Content and SEO	Article queue (existing), published articles and views, job hubs indexed vs noindexed, Search Console clicks and indexed pages, Core Web Vitals	Approve, edit, reject articles	P2

ID	Module	Shows	Actions	Priority
A17	LinkedIn	F1 adoption, PDF import success, More on LinkedIn clicks per weekly active user, notes generated, copied, sent, guard retries; monthly link-check reminder	None	P1
A18	System and security	Audit log, roles, kill switches, spend caps, data deletion requests, last backup and last test restore, error rate link to Sentry	Change flags, caps, roles (owner only)	P0

The full funnel

Every stage is counted per student once, in this order, and each step shows the share that carried on from the step before.

1. Visit: a landing or public page view, counted by Plausible with UTM tags.
2. Signup.
3. Profile ready: resume uploaded or LinkedIn import applied.
4. Preferences saved.
5. Alert channel connected: Telegram, WhatsApp or email confirmed. Steps 3 and 5 together are "activation".
6. First alert delivered.
7. First action: tailor, Email HR, Message on LinkedIn or Apply.
8. Trial started: mandate authorised.
9. First paid debit.
10. Renewed: second paid debit.
11. Upgraded to Pro + WhatsApp.

Breakdowns: source, medium, campaign, ambassador code, referral, college, city, device, and signup week. Date range defaults to the last 30 days.

Targets shown on the overview

Metric	Target	From
Signups	1,000 by day 60	Business plan
Activation	60% of signups	Business plan
Weekly active users	40% of signups	Business plan
Free → Pro conversion	10% within 60 days	Business plan
Autopay renewal success	85% of debits	Business plan
Tailored resumes per active user per week	2+	Business plan
LinkedIn F1 use	25% of new signups	LinkedIn spec
More on LinkedIn taps	15% of weekly active users	LinkedIn spec
Pro → Pro + WhatsApp upgrade	10% of Pro (proposed)	This spec
Organic share of signups	20% by month 3	SEO plan

Acceptance criteria

- A non-admin opening any `/admin` URL or `/api/admin/*` route gets 404.
- A Marketing user cannot see payments config or change a flag; the button is absent and the API returns 404.
- Turning off `whatsapp_sends` stops the next cron's WhatsApp sends, and the audit log shows who did it and when.
- The funnel's step counts match a manual SQL count for the same date range.
- A signup from `?utm_source=instagram&ref=PUNE-AMB-07` appears under Instagram and that ambassador.
- The overview loads in under 2 seconds with 10,000 users, because it reads daily rollups.
- No admin page or CSV shows a full phone number, resume text or an email list.

5. Admin panel TRD

The admin panel is a set of server-rendered pages under `/admin` in the same Next.js app, reading fast daily rollups and writing only through audited `/api/admin/*` routes.

Access

- `admin_users` holds `user_id` and `role` (`owner`, `ops`, `marketing`, `reviewer`). The first owner is inserted by hand in SQL; only owners can add others.
- `requireAdmin(roles[])` in `web/src/lib/admin/auth.ts` loads the session, reads the role from the database, and calls `notFound()` otherwise. The `/admin` layout calls it, and every page and route calls it again with its own roles.
- Admin writes use POST routes with zod and a same-origin check; no GET route changes data.
- `audit(actor, action, target, before, after)` in `web/src/lib/admin/audit.ts` is called inside the same transaction as the change.

Kill switches and spend caps

- `app_flags` rows: `whatsapp_sends`, `telegram_sends`, `email_sends`, `ai_generation`, `article_pipeline`, `paid_contact_lookups`, `signups`, `campaign_sends`. `getFlag(key)` caches for 30 seconds; senders and AI callers check it before each batch.
- `spend_caps` rows per provider (`whatsapp`, `anthropic`, `hunter`, `apollo`) hold a monthly cap in ₹ and an alert percentage (default 80). Spend is summed from `wa_messages.cost_inr` and the existing AI usage log. At the alert percentage, the owner gets one Telegram message; at 100%, the provider's sends stop until the cap is raised or the month ends.
- Admin alerts go to `ADMIN_TELEGRAM_CHAT_ID` through the existing bot: failed cron, spend at 80% and 100%, WhatsApp quality yellow or red, a template paused or moved to marketing, more than 10 webhook signature failures in an hour.

Data sources

Area	Source
Visits by source	Plausible (or Umami) on public pages only, with UTM tags; pulled daily into `metrics_daily`

Area	Source
Funnel steps 2–11	`events` rows written on the server at the moment each step happens
Attribution	`attributions`: first-touch UTM, referrer, ref code and landing path from a `gf_attr` cookie (30 days), saved at signup
Jobs, matches, tailoring, outreach, payments	Existing tables, aggregated by the rollup cron
Crons	`cron_runs`: every script writes start, finish, status, counts and error
WhatsApp health	`wa_health_snapshots`, filled daily from the Graph API (phone quality rating, messaging limit tier, template status and category)
SEO	`seo_daily`, filled daily from the Search Console API; Core Web Vitals weekly from the PageSpeed Insights API

Event names (server-side only, no personal data in `props`): `signup`, `profile_ready`, `prefs_saved`, `channel_connected`, `first_alert_delivered`, `first_action`, `trial_started`, `paid_first`, `renewed`, `upgraded_wa`, `downgraded_wa`, `cancelled`. Each "first" event is written once per user, guarded by a unique index on `(user_id, name)` for those names.

Rollups

- `web/scripts/admin/rollup.ts` runs at 02:00 IST and writes yesterday's numbers to `metrics_daily (day, metric, dim, value)`. The overview, funnel and cohorts read only this table plus today's live counts.
- Funnel queries count distinct users per event name within the date range, joined to `attributions` for breakdowns.
- Cohorts: share of each signup week active in each later week, where active means opened the app or acted on an alert.

Marketing features

- **Ambassadors:** `ambassadors` rows with a unique code such as `PUNE-AMB-07`. Links `/r/<code>` set the `gf_attr` cookie and redirect to the landing page.
- **Rewards and grants:** `pro_grants` gives Pro days without a mandate (ambassador rewards, referrals, support goodwill). Grants never include WhatsApp unless an owner ticks `includes_whatsapp`. Marketing can grant up to 30 days at a time.
- **Referrals:** each student gets a code. A referral qualifies on the referee's first paid debit; both get 15 Pro days. Cap: 10 rewards per referrer per month; same verified phone or same payment VPA as the referrer blocks it.
- **Campaigns:** email (Brevo or Resend) and Telegram only, never WhatsApp. Segments are fixed keys: `no_resume`, `no_channel`, `trial_ends_tomorrow`, `free_active_30d`, `cancelled_30d`, `halted`. Only users with `marketing_opt_in = true` are included; every email has an unsubscribe link. `campaign_recipients` makes a send idempotent; Telegram stays under 25 messages a second.
- **Telegram channel:** at 09:00 IST a cron drafts the day's post with the top 5 fresher jobs into `channel_posts`. Marketing can edit or skip until 09:30, then it posts. Links carry `utm_source=telegram_channel`.
- **Feedback:** an in-app "Send feedback" form writes `feedback`; admins tag it and mark it fixed.
- **Outcomes:** when a tracker card moves to Interview or Offer, an `events` row is written; the overview shows real counts only.

Admin API summary

Method + path	Roles	Does
POST `/api/admin/flags`	owner	Set a kill switch
POST `/api/admin/spend-caps`	owner	Set a cap
POST `/api/admin/roles`	owner	Add or remove an admin
POST `/api/admin/users/[id]/grant`	owner, ops, marketing	Grant Pro days (marketing ⩽ 30)
POST `/api/admin/users/[id]/reset-quota`	owner, ops	Reset this cycle's usage

Method + path	Roles	Does
POST `/api/admin/users/[id]/channel-off`	owner, ops	Turn off Telegram, email or WhatsApp
POST `/api/admin/users/[id]/disable`	owner, ops	Disable sign-in
POST `/api/admin/users/[id]/delete-data`	owner	Run the existing delete flow and log it
POST `/api/admin/jobs/[id]/hide`	owner, ops	Hide a job for all students
POST `/api/admin/boards`	owner, ops	Add or remove a company board
POST `/api/admin/crons/[name]/rerun`	owner, ops	Trigger the GitHub Actions workflow
POST `/api/admin/dnc`	owner, ops	Add or remove do-not-contact
POST `/api/admin/ambassadors`	owner, marketing	Create or deactivate a code
POST `/api/admin/referrals/[id]/block`	owner, marketing	Block a referral reward
POST `/api/admin/campaigns`	owner, marketing	Create, schedule or send
POST `/api/admin/channel-posts/[id]`	owner, marketing	Edit or skip
POST `/api/admin/feedback/[id]`	owner, ops, marketing	Tag or mark fixed
GET `/api/admin/export/funnel.csv`	owner, marketing	Aggregate counts only

Every route: `requireAdmin`, zod, general rate limit, `audit()` on success.

6. Database

Two migrations add 26 tables and 6 columns, all with row-level security; nothing existing is renamed or dropped. Migration A ships with PR 1 (admin foundation); migration B ships with PR 3 (WhatsApp).

Table	Change	Why
`admin_users`, `audit_log`, `app_flags`, `spend_caps`, `cron_runs`	New (`audit_log`, `cron_runs` if not already there)	Roles, accountability, switches, caps, cron health
`events`, `attributions`, `metrics_daily`, `seo_daily`	New	Funnel, acquisition, rollups, SEO
`plan_limits`, `pro_grants`	New	Limits from the database; Pro days without a mandate
`ambassadors`, `referrals`, `campaigns`, `campaign_recipients`, `channel_posts`, `feedback`, `job_reports`, `board_sources`, `deletion_requests`	New	Marketing and operations modules
`whatsapp_contacts`, `whatsapp_otps`, `wa_messages`, `wa_inbound`, `wa_rate_card`, `wa_health_snapshots`	New	WhatsApp
`users`	+ `marketing_opt_in`, `disabled_at`, `referral_code`; plan check gains `pro_wa`	Consent for campaigns, disable, referrals
`subscriptions`	+ `plan_code`	Pro vs Pro + WhatsApp
`jobs`	+ `hidden_at`, `hidden_reason`	Hide bad jobs

```
-- web/supabase/migrations/<ts>_admin_foundation.sql  (migration A)

create table public.admin_users (
  user_id    uuid primary key references auth.users(id) on delete cascade,
  role       text not null check (role in
('owner','ops','marketing','reviewer')),
  created_at timestamptz not null default now()
);
```

```
create table if not exists public.audit_log (
  id         bigserial primary key,
  actor_id   uuid references auth.users(id) on delete set null,
  action     text not null,
  target     text not null,
  before     jsonb,
  after      jsonb,
  created_at timestamptz not null default now()
);
create index if not exists audit_log_created_idx on public.audit_log
(created_at desc);

create table public.app_flags (
  key        text primary key,
  enabled    boolean not null default true,
  note       text,
  updated_by uuid references auth.users(id) on delete set null,
  updated_at timestamptz not null default now()
);
insert into public.app_flags (key) values
  ('whatsapp_sends'),('telegram_sends'),('email_sends'),('ai_generation'),
  ('article_pipeline'),('paid_contact_lookups'),('signups'),
('campaign_sends')
on conflict do nothing;

create table public.spend_caps (
  provider        text primary key check (provider in
('whatsapp','anthropic','hunter','apollo')),
  monthly_cap_inr numeric(10,2) not null,
  alert_pct       integer not null default 80 check (alert_pct between 1 and
100),
  alerted_at      timestamptz,
  stopped_at      timestamptz
);

create table if not exists public.cron_runs (
  id          bigserial primary key,
  job         text not null,
  started_at  timestamptz not null default now(),
  finished_at timestamptz,
  status      text not null default 'running' check (status in
('running','ok','warning','failed')),
```

```
  counts      jsonb not null default '{}'::jsonb,
  error       text
);
create index if not exists cron_runs_job_idx on public.cron_runs (job,
started_at desc);

create table public.events (
  id         bigserial primary key,
  name       text not null,
  user_id    uuid references auth.users(id) on delete set null,
  props      jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now()
);
create index events_name_time_idx on public.events (name, created_at);
create index events_user_idx on public.events (user_id);
create unique index events_first_once_uidx on public.events (user_id, name)
  where name in ('signup','profile_ready','prefs_saved','channel_connected',

'first_alert_delivered','first_action','trial_started','paid_first','renewed'
);

create table public.attributions (
  user_id       uuid primary key references auth.users(id) on delete cascade,
  utm_source    text, utm_medium text, utm_campaign text, utm_content text,
  referrer_host text,
  ref_code      text,
  landing_path  text,
  college       text,
  city          text,
  first_seen_at timestamptz
);
create index attributions_source_idx on public.attributions (utm_source,
utm_campaign);
create index attributions_ref_idx on public.attributions (ref_code);

create table public.metrics_daily (
  day    date not null,
  metric text not null,
  dim    text not null default 'all',
  value  numeric not null,
  primary key (day, metric, dim)
);
```

```
create table public.seo_daily (
  day           date primary key,
  clicks        integer, impressions integer,
  indexed_pages integer,
  cwv_good_pct  numeric(5,2)
);

create table public.plan_limits (
  plan                  text primary key check (plan in
('free','pro','pro_wa')),
  price_paise           integer not null,
  daily_alerts          boolean not null,
  whatsapp              boolean not null,
  tailor_per_cycle      integer not null,
  li_messages_per_cycle integer not null
);
insert into public.plan_limits values
  ('free',   0,    false, false, 1,  3),
  ('pro',    1000, true,  false, 10, 30),
  ('pro_wa', 2900, true,  true,  10, 30)
on conflict (plan) do nothing;

create table public.pro_grants (
  id                uuid primary key default gen_random_uuid(),
  user_id           uuid not null references auth.users(id) on delete
cascade,
  days              integer not null check (days between 1 and 365),
  includes_whatsapp boolean not null default false,
  reason            text not null check (reason in
('ambassador','referral','support','other')),
  granted_by        uuid references auth.users(id) on delete set null,
  starts_at         timestamptz not null default now(),
  ends_at           timestamptz not null
);
create index pro_grants_user_idx on public.pro_grants (user_id, ends_at
desc);

create table public.ambassadors (
  id         uuid primary key default gen_random_uuid(),
  user_id    uuid references auth.users(id) on delete set null,
  name       text not null,
  college    text not null,
  city       text not null,
```

```
  code       text not null unique check (code ~ '^[A-Z0-9-]{4,24}$'),
  active     boolean not null default true,
  created_at timestamptz not null default now()
);

alter table public.users
  add column marketing_opt_in boolean not null default false,
  add column disabled_at timestamptz,
  add column referral_code text unique;

create table public.referrals (
  id           uuid primary key default gen_random_uuid(),
  referrer_id  uuid not null references auth.users(id) on delete cascade,
  referee_id   uuid not null unique references auth.users(id) on delete
cascade,
  status       text not null default 'pending' check (status in
('pending','qualified','rewarded','blocked')),
  qualified_at timestamptz,
  rewarded_at  timestamptz,
  created_at   timestamptz not null default now(),
  check (referrer_id <> referee_id)
);

create table public.campaigns (
  id           uuid primary key default gen_random_uuid(),
  channel      text not null check (channel in ('email','telegram')),
  segment      text not null check (segment in
('no_resume','no_channel','trial_ends_tomorrow',

'free_active_30d','cancelled_30d','halted')),
  subject      text,
  body         text not null check (char_length(body) <= 4000),
  status       text not null default 'draft' check (status in
('draft','scheduled','sending','sent','cancelled')),
  scheduled_at timestamptz,
  sent_count   integer not null default 0,
  created_by   uuid references auth.users(id) on delete set null,
  created_at   timestamptz not null default now()
);
create table public.campaign_recipients (
  campaign_id uuid not null references public.campaigns(id) on delete
cascade,
  user_id     uuid not null references auth.users(id) on delete cascade,
```

```
  status      text not null default 'queued' check (status in
('queued','sent','failed','skipped')),
  sent_at     timestamptz,
  primary key (campaign_id, user_id)
);

create table public.channel_posts (
  id                  uuid primary key default gen_random_uuid(),
  post_date           date not null unique,
  body                text not null check (char_length(body) <= 4096),
  job_ids             uuid[] not null default '{}',
  status              text not null default 'draft' check (status in
('draft','skipped','posted','failed')),
  telegram_message_id bigint,
  updated_by          uuid references auth.users(id) on delete set null
);

create table public.feedback (
  id         uuid primary key default gen_random_uuid(),
  user_id    uuid references auth.users(id) on delete set null,
  body       text not null check (char_length(body) <= 2000),
  tags       text[] not null default '{}',
  status     text not null default 'new' check (status in
('new','planned','fixed','wontfix')),
  created_at timestamptz not null default now()
);

alter table public.jobs
  add column hidden_at timestamptz,
  add column hidden_reason text check (hidden_reason in
('scam','expired','duplicate','other'));

create table public.job_reports (
  id         uuid primary key default gen_random_uuid(),
  job_id     uuid not null references public.jobs(id) on delete cascade,
  user_id    uuid references auth.users(id) on delete set null,
  reason     text not null check (reason in
('scam','expired','wrong_info','other')),
  created_at timestamptz not null default now(),
  unique (job_id, user_id)
);

create table public.board_sources (
```

```
  id           uuid primary key default gen_random_uuid(),
  provider     text not null check (provider in
('greenhouse','lever','ashby')),
  slug         text not null,
  company_name text not null,
  active       boolean not null default true,
  unique (provider, slug)
);

create table public.deletion_requests (
  id           uuid primary key default gen_random_uuid(),
  email_hash   text not null,
  requested_by text not null check (requested_by in ('user','admin')),
  requested_at timestamptz not null default now(),
  completed_at timestamptz
);

-- RLS: own-row reads only where students need them; admin tables have no
-- policies, so only the service role (server routes after requireAdmin) can
use them.
alter table public.admin_users         enable row level security;
alter table public.audit_log           enable row level security;
alter table public.app_flags           enable row level security;
alter table public.spend_caps          enable row level security;
alter table public.cron_runs           enable row level security;
alter table public.events              enable row level security;
alter table public.attributions        enable row level security;
alter table public.metrics_daily       enable row level security;
alter table public.seo_daily           enable row level security;
alter table public.plan_limits         enable row level security;
alter table public.pro_grants          enable row level security;
alter table public.ambassadors         enable row level security;
alter table public.referrals           enable row level security;
alter table public.campaigns           enable row level security;
alter table public.campaign_recipients enable row level security;
alter table public.channel_posts       enable row level security;
alter table public.feedback            enable row level security;
alter table public.job_reports         enable row level security;
alter table public.board_sources       enable row level security;
alter table public.deletion_requests   enable row level security;

create policy "plan limits readable" on public.plan_limits for select to
authenticated using (true);
```

```
create policy "own grants"    on public.pro_grants for select using (user_id
= auth.uid());
create policy "own referrals" on public.referrals  for select using
(referrer_id = auth.uid());
```

```
-- web/supabase/migrations/<ts>_whatsapp.sql  (migration B)

-- Check the real name of the existing plan check constraint first.
alter table public.users drop constraint if exists users_plan_check;
alter table public.users add constraint users_plan_check check (plan in
('free','pro','pro_wa'));

alter table public.subscriptions
  add column plan_code text not null default 'pro' check (plan_code in
('pro','pro_wa'));

create table public.whatsapp_contacts (
  user_id            uuid primary key references auth.users(id) on delete
cascade,
  phone_e164         text not null check (phone_e164 ~ '^\+[1-9][0-9]
{7,14}$'),
  status             text not null default 'pending' check (status in
('pending','verified')),
  consent_at         timestamptz not null,
  consent_version    text not null,
  verified_at        timestamptz,
  opted_out_at       timestamptz,
  paused             boolean not null default false,
  fail_streak        integer not null default 0,
  last_auto_reply_at timestamptz,
  updated_at         timestamptz not null default now()
);
create unique index whatsapp_verified_phone_uidx on public.whatsapp_contacts
(phone_e164)
  where status = 'verified';

create table public.whatsapp_otps (
  id         uuid primary key default gen_random_uuid(),
  user_id    uuid not null references auth.users(id) on delete cascade,
  phone_e164 text not null,
  code_hash  text not null,
  attempts   integer not null default 0,
  expires_at timestamptz not null,
```

```
  created_at timestamptz not null default now()
);
create index whatsapp_otps_user_idx on public.whatsapp_otps (user_id,
created_at desc);

create table public.wa_rate_card (
  category       text not null check (category in
('marketing','utility','authentication','service')),
  inr_per_msg    numeric(8,4) not null,
  gst_pct        numeric(5,2) not null default 18,
  effective_from date not null,
  primary key (category, effective_from)
);
insert into public.wa_rate_card (category, inr_per_msg, effective_from)
values
  ('marketing', 0.8631, '2026-10-01'),
  ('utility', 0.1150, '2026-10-01'),
  ('authentication', 0.1150, '2026-10-01'),
  ('service', 0.1150, '2026-10-01')
on conflict do nothing;

create table public.wa_messages (
  id              uuid primary key default gen_random_uuid(),
  user_id         uuid references auth.users(id) on delete set null,
  kind            text not null check (kind in
('digest','verify','stop_confirm','auto_reply')),
  template        text,
  send_date       date not null,
  meta_message_id text unique,
  status          text not null default 'queued'
                  check (status in
('queued','sent','delivered','read','failed')),
  category        text,
  billable        boolean,
  cost_inr        numeric(10,4) not null default 0,
  error_code      text,
  created_at      timestamptz not null default now(),
  updated_at      timestamptz not null default now()
);
create unique index wa_one_digest_per_day_uidx on public.wa_messages
(user_id, send_date)
  where kind = 'digest';
create index wa_messages_date_idx on public.wa_messages (send_date, status);
```

```
create table public.wa_inbound (
  id              uuid primary key default gen_random_uuid(),
  user_id         uuid references auth.users(id) on delete set null,
  phone_hash      text not null,
  body            text check (char_length(body) <= 1000),
  keyword         text check (keyword in ('stop','start')),
  meta_message_id text not null unique,
  received_at     timestamptz not null default now()
);

create table public.wa_health_snapshots (
  id                   uuid primary key default gen_random_uuid(),
  quality_rating       text,
  messaging_limit_tier text,
  templates            jsonb not null default '[]'::jsonb,
  captured_at          timestamptz not null default now()
);

alter table public.whatsapp_contacts   enable row level security;
alter table public.whatsapp_otps       enable row level security;
alter table public.wa_rate_card        enable row level security;
alter table public.wa_messages         enable row level security;
alter table public.wa_inbound          enable row level security;
alter table public.wa_health_snapshots enable row level security;

create policy "own whatsapp contact" on public.whatsapp_contacts for select
using (user_id = auth.uid());
-- No other policies: all writes and every other read go through server
routes with the service role.
```

Notes for Claude Code:

- Confirm the existing `users.plan` check constraint name before running migration B, and that `alert_settings.channels` accepts `whatsapp`.
- Seed `spend_caps` from the owner's values in the admin UI after deploy; there is no default cap, and WhatsApp sends stay off until a WhatsApp cap exists.
- "Delete my account" must also delete `whatsapp_contacts`, `whatsapp_otps` and `wa_inbound` rows, null `user_id` in `events` and `wa_messages`, and write a `deletion_requests` row with only a hash of the email.
- `wa_inbound` rows older than 90 days are deleted by the daily cleanup cron.

- Regenerate Supabase TypeScript types after each migration.

7. Security, compliance and tests

Every new route follows plan section 7; the table lists only what is new for WhatsApp and the admin panel.

Risk	Fix	Test
Forged WhatsApp webhook	HMAC-SHA256 of the raw body with the app secret, constant-time compare, before parsing	Unsigned or wrongly signed POST: 401, no row changed
Cost runaway (cron bug, retries)	Unique digest per user per IST day; `whatsapp_sends` switch; monthly cap enforced in `client.ts`	Run the sender twice: one message per user; cap reached: zero sends
Sending to someone who did not ask	Verified number, versioned consent, STOP honoured at once	Opted-out user gets nothing next morning
Number takeover	Code sent to the number itself; one verified account per number	Second account verifying a taken number: 409
OTP brute force	Hash only, 3 tries, 10-minute expiry, 3 sends per hour	Fourth wrong code: locked until a new code
Free user gets WhatsApp	Entitlement read from `plan_limits` and subscriptions in `canSendWhatsApp`	Free user calling verify/start directly: 403
Template parameter injection	Strip newlines and tabs, cut to 60 characters, plain text only	Job title with newlines and 200 characters sends a clean line
Meta token leak	Server env only; CI grep fails on `WHATSAPP_TOKEN` in client code or logs	CI step

Risk	Fix	Test
Phone numbers exposed	`maskPhone()` in admin and logs; inbound stores a hash of the number; no bulk export	Admin user page shows 98xxxxxx21
Admin privilege escalation	Role read from `admin_users` on every request; 404 for others; owner-only routes for flags, caps, roles	Ops user posting to `/api/admin/flags`: 404
Admin CSRF	POST only, same-origin check, SameSite cookies	Cross-origin POST: 403
Untraceable admin changes	`audit()` in the same transaction as the change	Toggle a flag: one audit row with before and after
Referral fraud	Qualify only on first paid debit; 10 rewards per month; block same phone or VPA	Self-referral with a second email: no reward
Spam campaigns	Opt-in only, unsubscribe link, idempotent recipients, `campaign_sends` switch	Opted-out user not in recipients
Analytics personal data	No email, phone or resume text in `events.props`; aggregate-only CSV	CSV contains no email column
DPDP deletion	Delete flow covers new WhatsApp and marketing tables; hashed deletion log	Delete account: no WhatsApp rows left, events have null user

Tests Claude Code must add:

- Unit: `phone.ts` (valid Indian number, +91 missing, letters, too long) and `maskPhone`.
- Unit: template parameter sanitiser and template choice for 1, 2, 3 and 5 matches.
- Unit: `canSendWhatsApp` for each failing condition and the passing case.
- Unit: cost calculation for each category with GST, from the rate card.
- Unit: webhook signature check with valid, invalid and missing headers.
- Route: every new route returns 401 without a session; every admin route returns 404 for a non-admin and for the wrong role.
- Route: verify start, confirm, wrong code three times, number already taken.

- Route: webhook STOP, START, other text (one auto-reply per 24 hours), duplicate event ignored, delivered and failed statuses.
- Route: Razorpay `pro_wa` authenticated, activated, charged, halted, cancelled.
- Job: notify cron twice in one IST day sends one digest; no matches sends none; switch off or cap reached sends none.
- Admin: flag toggle writes an audit row; grant above 30 days by Marketing is refused.
- Funnel: fixture data gives the expected step counts and breakdown by source and ambassador.
- E2E (Playwright, mocked Meta and Razorpay): Pro user upgrades, verifies, sees "WhatsApp connected", receives a mocked digest.

8. UI: DESIGN.md additions to approve

DESIGN.md does not cover WhatsApp or admin screens yet, so these proposals need your yes before Claude Code builds any UI. Each keeps the existing rules: tokens only, one primary button per view, badges limited to the three allowed, Lucide icons only.

Decision	Proposal	Why
WhatsApp icon	No brand logo. Write "WhatsApp"; use Lucide `message-circle` only where an icon is needed	Lucide-only rule
Pricing block	Keep two plans. The Pro card gets a checkbox row "Add WhatsApp alerts · + ₹19" that changes the price to "₹29 every 15 days"	The two-plan layout stays calm
Upgrade entry	On Alerts and Billing: secondary button "Add WhatsApp alerts" for Pro users; hidden for Free users, who see it inside the Pro card	One primary per view
Connect flow	Same framed panel as Connect Telegram: phone field with +91, consent checkbox with the exact consent copy, primary "Send code", then a 6-digit code field and primary "Verify"	Reuses an existing pattern

Decision	Proposal	Why
Connected state	Same chip as Telegram: grey "WhatsApp not connected" → success "WhatsApp connected" with check; paused shows warning "WhatsApp paused" with the fix	Existing feedback pattern
Billing page	Plan line "Pro + WhatsApp · ₹29 every 15 days"; next debit in large type; a text button "Switch to Pro from next cycle"	Money copy is explicit
Admin shell	240 px left sidebar grouped as Overview, Operations, Money, Growth, Content, System; desktop-first, usable at 768 px; no mobile tabs	Admin is used at a desk
Admin numbers	Large tabular figures with the target beside them in ink-3: "412 signups · target 1,000 by day 60". Colour only when off target: warning, then danger	Real numbers, quiet by default
Admin tables	Flat rows, 1 px line borders, sticky header, sortable columns, 50 rows per page, masked phones	Same flat-row style as JobCard
Admin charts	Recharts, restyled with tokens, lazy-loaded only in `/admin` so landing JS stays under 170 KB. Line for trends, bars for the funnel; no pie charts	Performance budget
Kill switches	Danger text buttons; confirm dialog asks the admin to type the switch name; toast "WhatsApp sends paused"	Hard to press by accident
Admin empty states	"No data yet" plus one line on when data appears, e.g. "Filled after tonight's rollup at 02:00 IST."	Empty-state rule

Copy to add to the DESIGN.md "Use" list: Add WhatsApp alerts, WhatsApp connected, Send code, Verify, Switch to Pro from next cycle. Toasts: "Code sent", "WhatsApp connected", "WhatsApp alerts off", "Saved", "Sends paused", "Sends resumed".

Proposed block to paste into DESIGN.md after the LinkedIn section:

```
## WhatsApp
- No WhatsApp logo. Write "WhatsApp"; Lucide message-circle only where an
icon is needed.
- Pricing: two plans stay. Pro card has a checkbox row "Add WhatsApp alerts ·
+₹19" that switches the price to "₹29 every 15 days".
- Connect panel: like Connect Telegram. +91 phone field, consent checkbox
with the exact consent copy, primary "Send code", then a 6-digit field and
primary "Verify".
- Chip: grey "WhatsApp not connected" → success "WhatsApp connected"; warning
"WhatsApp paused" with the fix.

## Admin
- Desktop-first shell, 240 px sidebar: Overview, Operations, Money, Growth,
Content, System.
- Numbers in tabular figures with the target in ink-3 beside them; warning or
danger colour only when off target.
- Flat tables with sticky headers, sortable, 50 rows per page; phones always
masked.
- Charts: Recharts with tokens, lazy-loaded in /admin only; lines for trends,
bars for the funnel, no pies.
- Kill switches: danger text buttons with a type-to-confirm dialog.
- Empty: "No data yet" plus when data will appear.
```

9. Build order and Claude Code prompts

Build in 9 small PRs, about 12–15 working days in total; each prompt below is one PR. PRs 1–2 ship before the public launch, PRs 3–5 before WhatsApp goes live, and PRs 6–9 in the weeks after.

1. **Admin foundation.** Migration A, `requireAdmin`, audit, flags, spend caps, cron_runs logging, admin alerts, admin shell and overview.
2. **Users, payments, system.** User lookup and actions, payments module, do-not-contact, system page.
3. **WhatsApp core.** Migration B, WhatsApp lib, verification routes, webhook. No digest sending yet.
4. **WhatsApp plan.** Razorpay `pro_wa`, upgrade and downgrade, entitlement, pricing and settings UI.
5. **WhatsApp digest and admin page.** Daily sender in the notify cron, health snapshot cron, admin WhatsApp module.

6. **Operations modules.** Jobs, matching, alerts, AI, tailoring, outreach, LinkedIn admin pages.
7. **Funnel and acquisition.** Events, attribution cookie, rollups, funnel page, ambassadors, referrals, Pro grants.
8. **Growth tools.** Campaigns, Telegram channel scheduler, cohorts, feedback, outcomes.
9. **Content, SEO and audit.** Content and SEO pages, then the security audit prompt.

PR 1 prompt:

```
Read CLAUDE.md, DESIGN.md and docs/whatsapp-admin/SPEC.md. Implement PR 1
only:
migration A from section 6, regenerate Supabase types, then build
web/src/lib/admin/auth.ts (requireAdmin), audit.ts, flags.ts (getFlag with a
30-second cache) and spend.ts, a cron_runs wrapper that every existing script
in web/scripts uses, and admin Telegram alerts to ADMIN_TELEGRAM_CHAT_ID.
Build
the /admin layout with the sidebar from the DESIGN.md Admin section and the
Overview page (targets from section 4, problems list). Add the flags and
spend-caps routes for owners. Add the admin auth and audit tests from
section 7. Run npm run lint and npm run build. List anything in the spec you
could not follow.
```

PR 2 prompt:

```
Implement PR 2 from docs/whatsapp-admin/SPEC.md: modules A2 Users, A9 HR
outreach (do-not-contact part), A11 Payments and A18 System and security,
with the admin API routes they need from section 5. Phones always masked;
no resume text on any page. Every write uses requireAdmin with the roles in
section 4 and calls audit(). Follow the DESIGN.md Admin section; tokens only;
check 768 and 1440 px. Add route tests for 404 on wrong roles.
```

PR 3 prompt:

```
Implement PR 3 from docs/whatsapp-admin/SPEC.md: migration B from section 6,
regenerate types, web/src/lib/whatsapp/* (client, phone, templates, cost,
eligibility), the verify start and confirm routes, the disconnect route and
the webhook with signature checking, STOP and START handling and the
one-per-24-hours auto-reply. client.ts is the only file allowed to call
graph.facebook.com and must check the whatsapp_sends flag and the spend cap.
Do not send digests yet. Add every WhatsApp unit and route test in section 7.
```

PR 4 prompt:

```
Implement PR 4 from docs/whatsapp-admin/SPEC.md (W3). Use the pro_wa
Razorpay plan id from env, build the upgrade and downgrade routes, handle
pro_wa events in the existing Razorpay webhook as section 3 describes, and
resolve the effective plan from subscriptions plus pro_grants. Read all
limits
from plan_limits. Build the pricing checkbox row, the Add WhatsApp alerts
entry, the connect panel and the connected chip per the DESIGN.md WhatsApp
section. Add the Razorpay route tests and the Playwright upgrade test.
```

PR 5 prompt:

```
Implement PR 5 from docs/whatsapp-admin/SPEC.md: sendWhatsAppDigest in
web/scripts/notify/whatsapp.ts, called by the existing notify cron for users
whose channels include whatsapp, with the one-per-IST-day unique row,
template
choice, 20 messages per second and fail_streak pausing. Add the daily
whatsapp-health script. Build admin module A6 WhatsApp from section 4. Add
the
"cron twice sends one digest", "no matches sends none" and "switch off sends
none" tests.
```

PR 6 prompt:

```
Implement PR 6 from docs/whatsapp-admin/SPEC.md: admin modules A3 Jobs and
ingestion (including board_sources and job hiding, and making the fetchers
read board_sources and skip hidden jobs), A4 Matching, A5 Alerts and crons
(with rerun through the GitHub Actions workflow_dispatch API), A7 AI usage
and cost, A8 Resume tailoring, the rest of A9 HR outreach, and A17 LinkedIn.
Read from metrics_daily where the section says so.
```

PR 7 prompt:

```
Implement PR 7 from docs/whatsapp-admin/SPEC.md: the gf_attr cookie and
/r/<code>
redirect, attributions saved at signup, server-side events for every funnel
step in section 5, the 02:00 IST rollup script into metrics_daily, the
Plausible
daily pull, admin module A12 Funnel and acquisition with every breakdown in
section 4, A13 Ambassadors and referrals with the fraud rules, and
pro_grants.
Add the funnel fixture test and the referral fraud test.
```

PR 8 prompt:

```
Implement PR 8 from docs/whatsapp-admin/SPEC.md: A14 campaigns (email and
Telegram only, fixed segments, marketing_opt_in, unsubscribe, idempotent
recipients, campaign_sends flag) and the Telegram channel post cron with the
09:00 draft and 09:30 post, A15 retention cohorts and the feedback form and
inbox, and A10 outcomes. Add a marketing opt-in checkbox to Settings,
unticked
by default.
```

PR 9 prompt:

```
Implement A16 Content and SEO from docs/whatsapp-admin/SPEC.md, including the
Search Console and PageSpeed daily pulls into seo_daily. Then act as a senior
application security engineer: audit all WhatsApp and admin code against
section 7 of docs/whatsapp-admin/SPEC.md and section 7 of the plan. Report
pass
or fail with file and line, fix failures, confirm only client.ts calls
graph.facebook.com, and confirm no admin page or CSV exposes phone numbers,
emails or resume text.
```

After WhatsApp goes live, check the rate card on the 1st of each month and add a new `wa_rate_card` row when Meta changes prices; never edit old rows.

10. Content quality bot

A quality bot scores every AI-written article on 10 checks and auto-publishes only low-risk articles that pass all of them; news about named companies still needs a one-click human approval. This changes plan section 6 ("nothing auto-publishes") for the green tier only, and ships as PR 10 after PR 9.

Hard rules

- The bot makes articles clearer and more useful. It never tunes text to get past AI detectors; Google does not rank on that, and it penalises scaled low-value content however it was made.
- Rewrites may change wording only. The fact check and overlap check run again after every rewrite.
- Auto-publish only the green tier. Yellow waits for a human; red is rejected with reasons.

- At most 3 articles published per day in total. The `article_pipeline` switch stops everything; a new `article_autopublish` switch stops only auto-publishing and sends every passing draft to the queue.
- Byline: auto-published articles say "GeeksFinder Desk"; human-approved ones say "GeeksFinder Desk, reviewed by <name>". The editorial policy page explains both.

Pipeline

The bot runs after step 4 ("Write") of the plan's article pipeline and replaces step 5:

1. Code checks: overlap, facts, readability, word list, SEO rules, safety flags.
2. If readability or word list fails: a plain-language pass rewrites only the failing sentences. At most 2 rounds.
3. Run the overlap and fact checks again on the rewritten text.
4. Web originality check through a paid plagiarism API.
5. Judge model scores the article.
6. SEO auto-fixes (meta description, slug, alt text), then the SEO rules again.
7. Decide the tier, then publish (green), queue (yellow) or reject (red). Red drafts get one full rewrite attempt before rejection.

Checks

Check	Rule	On failure
Overlap with sources	No run of 8 or more words shared with any source (existing)	Rewrite those sentences; red if still failing
Web originality	Plagiarism API similarity of 10% or less with any page (proposed threshold)	Red
Own-site duplicate	Under 50% shingle similarity with our articles from the last 90 days; the same company and event in the last 7 days becomes an update to that article	Red, or turn into an update

Check	Rule	On failure
Facts	Every number, name and date is in the fact list (existing); every quote appears word for word in a source; 2 or more sources on different domains that are not copies of one press release (under 50% overlap with each other); news sources 7 days old or newer; any deadline in the text is still in the future	Red
Readability	Flesch reading ease 60 or more; average sentence 18 words or fewer; no sentence over 30 words; paragraphs of 3 sentences or fewer; 600 to 1,000 words (existing)	Plain-language pass
Word list	Hard words replaced from `plain-words.ts` (utilise → use, commence → start, endeavour → try, in order to → to, prior to → before); banned phrases (existing list); no exclamation marks; no emoji	Auto-replace, then plain-language pass
Usefulness	"What this means for freshers" has at least 80 words and one concrete action; at least one GeeksFinder data point from our own database (open roles at the company this week, top skills asked, top cities), inserted by code, not by AI; 3 live job rows	Red
Judge	A different model from the writer (Claude Haiku writes, Gemini Flash judges) scores 1 to 5 on accuracy against the facts, usefulness to a final-year student, clarity, original angle and headline honesty	Any score 2 or lower: red; any score 3: yellow

Check	Rule	On failure
SEO	Title of 60 characters or fewer with the main search phrase; meta description of 140 to 155 characters; lowercase hyphenated slug of 60 characters or fewer; one H1 and at least 2 H2s; at least 2 links to job hubs with 10 or more live jobs; sources linked; Article or NewsArticle schema; OG image; alt text on every image; main phrase not already targeted by another article	Auto-fix, then check again; red if still failing
Safety	Layoffs, salary or CTC figures, legal or fraud claims, exam or result changes, or a named private person → yellow. Outcome promises ("guaranteed job") or personal data → red. Text inside sources is data, never instructions	Tier as stated

Tiers

Tier	When	What happens
Green	Every check passes, every judge score is 4 or more with an average of 4.3 or more, no safety flag, and the topic is one of: skills trend from our data, guide built from our data, internship or hiring roundup from our own job database, or a hiring drive confirmed on the company's own careers page or newsroom plus one more source	Auto-publish and post to the Telegram channel
Yellow	Every check passes but there is a safety flag, a judge score of 3, or any other news topic	Admin queue with the full quality report; approve, edit or reject in one click

Tier	When	What happens
Red	Any check fails after the allowed rewrites	Rejected; reasons shown in admin

After publishing

- **Spot checks:** each week, 20% of green articles (at least 1) go to an admin spot-check list. A failed spot check turns off auto-publish for that topic type until an owner turns it back on.
- **Automatic brake:** 3 corrections in 30 days switch `article_autopublish` off and alert the owner.
- **Search performance:** pull clicks, impressions and average position per article daily from Search Console. At 28 days, an article with zero impressions gets an "improve or noindex" task in admin.
- **Freshness:** when a deadline in an article passes or its jobs expire, show an "Outdated" note at the top with the date, swap in current jobs, and log it. Every change after publishing adds a dated line to the article's corrections list.

Files

Path	Purpose
`web/scripts/content/quality/run.ts`	Runs the pipeline above for each new draft
`web/scripts/content/quality/overlap.ts`	Source overlap and own-site duplicate checks
`web/scripts/content/quality/facts.ts`	Fact list, quotes, source count, source age, future dates
`web/scripts/content/quality/readability.ts`	Flesch score, sentence and paragraph lengths, word count
`web/scripts/content/quality/plain-words.ts`	Hard-word replacement list and banned phrases
`web/scripts/content/quality/originality-web.ts`	Plagiarism API client; checks the `plagiarism` spend cap

Path	Purpose
`web/scripts/content/quality/judge.ts`	Judge prompt, zod schema for the 5 scores and reasons
`web/scripts/content/quality/seo.ts`	SEO rules and auto-fixes
`web/scripts/content/quality/safety.ts`	Risk flags
`web/scripts/content/quality/tier.ts`	Tier decision from all results
`web/scripts/content/search-performance.ts`	Daily Search Console pull per article

Database

```
-- web/supabase/migrations/<ts>_content_quality.sql

alter table public.article_drafts
  add column tier           text check (tier in ('green','yellow','red')),
  add column quality_score  numeric(3,2),
  add column risk_flags     text[] not null default '{}',
  add column web_similarity numeric(5,2),
  add column rewrite_rounds integer not null default 0,
  add column decided_by     text check (decided_by in ('bot','human'));

alter table public.articles
  add column auto_published  boolean not null default false,
  add column reviewed_by     uuid references auth.users(id) on delete set
null,
  add column topic_type      text,
  add column outdated_at     timestamptz,
  add column corrections     jsonb not null default '[]'::jsonb,
  add column last_checked_at timestamptz;

create table public.article_checks (
  id         uuid primary key default gen_random_uuid(),
  draft_id   uuid not null references public.article_drafts(id) on delete
cascade,
  check_name text not null,
  round      integer not null default 0,
  passed     boolean not null,
  score      numeric,
  details    jsonb not null default '{}'::jsonb,
```

```
  created_at timestamptz not null default now()
);
create index article_checks_draft_idx on public.article_checks (draft_id,
created_at);

create table public.article_search_daily (
  article_id   uuid not null references public.articles(id) on delete
cascade,
  day          date not null,
  clicks       integer not null default 0,
  impressions  integer not null default 0,
  avg_position numeric(6,2),
  primary key (article_id, day)
);

create table public.article_spot_checks (
  id          uuid primary key default gen_random_uuid(),
  article_id  uuid not null references public.articles(id) on delete cascade,
  status      text not null default 'pending' check (status in
('pending','passed','failed')),
  note        text,
  reviewer_id uuid references auth.users(id) on delete set null,
  created_at  timestamptz not null default now()
);

insert into public.app_flags (key) values ('article_autopublish') on conflict
do nothing;

alter table public.spend_caps drop constraint if exists
spend_caps_provider_check;
alter table public.spend_caps add constraint spend_caps_provider_check
  check (provider in
('whatsapp','anthropic','hunter','apollo','plagiarism'));

alter table public.article_checks       enable row level security;
alter table public.article_search_daily enable row level security;
alter table public.article_spot_checks  enable row level security;
-- No policies: admin and cron access only, through the service role.
```

Admin additions (module A16)

- Each draft in the queue shows its tier, every check with pass or fail and the reason, the judge's 5 scores, web similarity and the diff of each rewrite round.
- New lists: auto-published articles, spot checks due, corrections, "improve or noindex" tasks.
- Owner-only `article_autopublish` switch and the per-topic-type auto-publish toggles, both audited.

Tests

- Unit: each check with a passing and a failing fixture article.
- Unit: the plain-language pass never changes a number, name or date (fact check runs again and passes).
- Unit: tier decision for every row of the tiers table, including a layoffs article with perfect scores going to yellow.
- Unit: own-site duplicate turns a same-event article into an update.
- Job: `article_autopublish` off sends a green draft to the queue; 3 corrections in 30 days turn it off.
- Job: the 3-per-day cap holds when 5 green drafts arrive.

PR 10 prompt:

```
Read CLAUDE.md, DESIGN.md and section 10 of docs/whatsapp-admin/SPEC.md.
Implement the content quality bot: the migration, every file in the Files
table, the pipeline order, the checks with the exact thresholds, the tier
rules, the spot checks, the automatic brake, the Search Console pull and the
A16 admin additions. The plain-language pass may only rewrite failing
sentences and must re-run the fact and overlap checks. Never add code that
scores or rewrites text against AI detectors. Respect article_pipeline,
article_autopublish and the plagiarism spend cap. Add every test in section
10. Run npm run lint and npm run build, and list anything you could not
follow.
```